

Sequelize ORM Query Chart

Operation	Query (Sequelize)	Description
 Create	<code>Model.create(data)</code>	Insert a single record
	<code>Model.bulkCreate(dataArray)</code>	Insert multiple records
 Read / Find	<code>Model.findAll()</code>	Get all records
	<code>Model.findOne({ where: { email: "abc@example.com" } })</code>	Find one by condition
	<code>Model.findByPk(id)</code>	Find by primary key
	<code>Model.findAndCountAll()</code>	Paginated fetch with total count
	<code>Model.findAll({ include: [OtherModel] })</code>	Get with relations
 Filtering	<code>where: { email: { [Op.like]: "%@gmail.com" } }</code>	LIKE query (use Op)
	<code>where: { age: { [Op.gt]: 18 } }</code>	Greater than 18
	<code>where: { age: { [Op.between]: [20, 30] } }</code>	BETWEEN query
 Update	<code>Model.update(data, { where: condition })</code>	Update matching records
	<code>instance.save()</code>	Save updated model instance
 Delete	<code>Model.destroy({ where: condition })</code>	Delete matching records
	<code>instance.destroy()</code>	Delete a model instance
 Associations	<code>Model.hasMany(OtherModel) / belongsTo()</code>	Create associations
	<code>User.findAll({ include: Blog })</code>	Eager load relations
 Syncing	<code>sequelize.sync({ alter: true })</code>	Sync models to DB without dropping data
	<code>sequelize.sync({ force: true })</code>	Drops & recreates tables (dev only)
 Raw Query	<code>sequelize.query('SELECT * FROM users')</code>	Custom SQL queries
 Transactions	<code>sequelize.transaction(async (t) => { ... })</code>	Perform operations within transaction

Operation	Query (Sequelize)	Description
 Migrations (CLI)	<code>npx sequelize-cli migration:generate --name create-users</code>	Create migration file
	<code>npx sequelize-cli db:migrate</code>	Apply migrations

Sequelize Query Methods Cheat Sheet

1. CREATE

- **create()**: Creates a single record and returns the created instance.

js

CopyEdit

```
const user = await User.create({
  email: 'john@example.com',
  password: 'password123'
});
```

- **bulkCreate()**: Creates multiple records in a single query.

js

CopyEdit

```
const users = await User.bulkCreate([
  { email: 'user1@example.com', password: 'password1' },
  { email: 'user2@example.com', password: 'password2' }
]);
```

2. READ

- **findOne()**: Fetches a single record by primary key or a query.

js

CopyEdit

```
const user = await User.findOne({
  where: { email: 'john@example.com' }
```

```
});
```

- **findAll()**: Fetches all records that match a query.

```
js
```

CopyEdit

```
const users = await User.findAll({  
  where: { isActive: true }  
});
```

- **findByPk()**: Fetches a record by its primary key.

```
js
```

CopyEdit

```
const user = await User.findByPk(1); // Find user with ID 1
```

- **findAndCountAll()**: Fetches records and the total count.

```
js
```

CopyEdit

```
const { count, rows } = await User.findAndCountAll({  
  where: { isActive: true }  
});
```

- **count()**: Returns the count of records based on a query.

```
js
```

CopyEdit

```
const count = await User.count({  
  where: { email: 'john@example.com' }  
});
```

3. UPDATE

- **update()**: Updates a record or multiple records. Returns the number of affected rows.

```
js
```

CopyEdit

```
const [updated] = await User.update(  
  { password: 'newpassword123' },  
  { where: { email: 'john@example.com' } }  
);
```

- **increment()**: Increments a field (e.g., like counter).

js

CopyEdit

```
await User.increment('loginCount', { by: 1, where: { id: 1 } });
```

- **decrement()**: Decrements a field (e.g., likes count).

js

CopyEdit

```
await User.decrement('loginCount', { by: 1, where: { id: 1 } });
```

4. DELETE

- **destroy()**: Deletes records matching the query.

js

CopyEdit

```
const deleted = await User.destroy({  
  where: { email: 'john@example.com' }  
});
```

- **truncate()**: Deletes all records in a table.

js

CopyEdit

```
await User.truncate();
```

5. ASSOCIATIONS

- **hasMany()**: Defines a one-to-many relationship.

js

CopyEdit

```
User.hasMany(Blog, { foreignKey: 'authorId' });
```

- **belongsTo()**: Defines the opposite of a one-to-many.

js

CopyEdit

```
Blog.belongsTo(User, { foreignKey: 'authorId' });
```

- **hasOne()**: Defines a one-to-one relationship.

js

CopyEdit

```
User.hasOne(Profile, { foreignKey: 'userId' });
```

- **belongsToMany()**: Many-to-many relationship.

js

CopyEdit

```
User.belongsToMany(Role, { through: 'UserRoles' });
```

```
Role.belongsToMany(User, { through: 'UserRoles' });
```

- **include**: Used to load associations (Eager Loading).

js

CopyEdit

```
const blogs = await Blog.findAll({  
  include: [{  
    model: User,  
    where: { email: 'john@example.com' }  
  }]  
});
```

6. TRANSACTIONS

- **transaction()**: Executes operations within a transaction.

js

CopyEdit

```
const t = await sequelize.transaction();  
  
try {  
  const user = await User.create({ email: 'john@example.com' }, { transaction: t });  
  const blog = await Blog.create({ title: 'My Blog', authorId: user.id }, { transaction: t });  
  await t.commit();  
} catch (err) {  
  await t.rollback();  
}
```

7. RAW QUERIES

- **sequelize.query()**: Executes a raw SQL query.

js

CopyEdit

```
const result = await sequelize.query('SELECT * FROM users WHERE email = ?', {
  replacements: ['john@example.com'],
  type: sequelize.QueryTypes.SELECT
});
```

8. DATABASE SYNC

- **sequelize.sync():** Syncs the models with the database (creates tables).

js

CopyEdit

```
sequelize.sync({ force: true }) // Drops tables and recreates
.then(() => console.log('Database synced'));
```

Summary Table

Operation	Method	Example / Syntax
Create	create()	await User.create({ email, password })
Bulk Create	bulkCreate()	await User.bulkCreate([{...}, {...}])
Read	findOne()	await User.findOne({ where: { email } })
	findAll()	await User.findAll({ where: { isActive } })
	findByPk()	await User.findByPk(1)
	findAndCountAll()	await User.findAndCountAll({ where: { status } })
	count()	await User.count({ where: { email } })
Update	update()	await User.update({ password }, { where: { email } })
	increment()	await User.increment('loginCount', { by: 1 })
	decrement()	await User.decrement('loginCount', { by: 1 })
Delete	destroy()	await User.destroy({ where: { email } })
	truncate()	await User.truncate()
Associations	hasMany()	User.hasMany(Blog, { foreignKey })
	belongsTo()	Blog.belongsTo(User, { foreignKey })
	hasOne()	User.hasOne(Profile, { foreignKey })

Operation	Method	Example / Syntax
	<code>belongsToMany()</code>	<code>User.belongsToMany(Role, { through: 'UserRoles' })</code>
Transactions	<code>transaction()</code>	Use transaction to wrap multiple operations
Raw Queries	<code>sequelize.query()</code>	<code>await sequelize.query('SELECT * FROM users')</code>
Sync DB	<code>sequelize.sync()</code>	<code>sequelize.sync({ force: true })</code>

This cheat sheet should give you a solid understanding of how to work with Sequelize in your projects! You can easily integrate it with your MySQL/PostgreSQL projects for various CRUD operations and associations.